PSEUDOCODE (SÖZDE KOD) VE PROBLEM ÇÖZME

1. GİRİŞ

Bir problemi çözmek için önce problemi anlamak, sonra çözüm adımlarını planlamak, ardından bu adımları bir programa dönüştürmek gerekir. İşte bu noktada **sözde kod (pseudocode)** devreye girer. Sözde kod, belirli bir programlama dilinin sözdizimi kurallarına bağlı kalmadan, algoritmanın okunabilir biçimde yazılmasını sağlar. Bu bölümde sözde kodun ne olduğu, algoritmik düşünme teknikleri ve matematiksel problemlerin sözde kod ile nasıl çözüleceği ayrıntılı biçimde ele alınacaktır.

2. SÖZDE KOD (PSEUDOCODE) NEDİR?

Sözde kod, bir algoritmanın programlama diline yakın ancak dilin katı kurallarına bağlı olmayan, insan tarafından kolayca okunabilen bir gösterim biçimidir.

2.1 Sözde Kodun Amaçları

1. Algoritmayı **dilden bağımsız** olarak ifade etmek.
2. Kodlamaya geçmeden önce **mantığı netleştirmek**.
3. Farklı programcılar arasında **ortak anlatım** sağlamak.
4. Hataları **kod yazmadan önce** tespit etmek.

2.2 Sözde Kodun Özellikleri

```
+----------------------------+-------------------------------------------+
| Özellik                    | Açıklama                                  |
+----------------------------+-------------------------------------------+
| Dilden bağımsız            | Herhangi bir programlama dili değildir    |
| Okunabilir                 | İnsan diliyle benzer yapıdadır            |
| Kısa ve öz                 | Gereksiz ayrıntı içermez                  |
| Yapısal                    | Sıralı, koşullu, döngülü yapılar içerir   |
| Kodlamaya rehber           | Program yazmadan önce yol gösterir        |
+----------------------------+-------------------------------------------+
```

2.3 Sözde Kod ile Gerçek Kod Arasındaki Fark

Sözde Kod	Gerçek Kod
Dil bağımsız	Belirli bir dile bağlı
İnsan odaklı	Makine odaklı
Esnek sözdizimi	Katı sözdizimi
Hızlı yazılır	Derlenme/yorumlanma gerekir

Sözde Kod	Gerçek Kod
Mantık ön planda	Uygulama ön planda

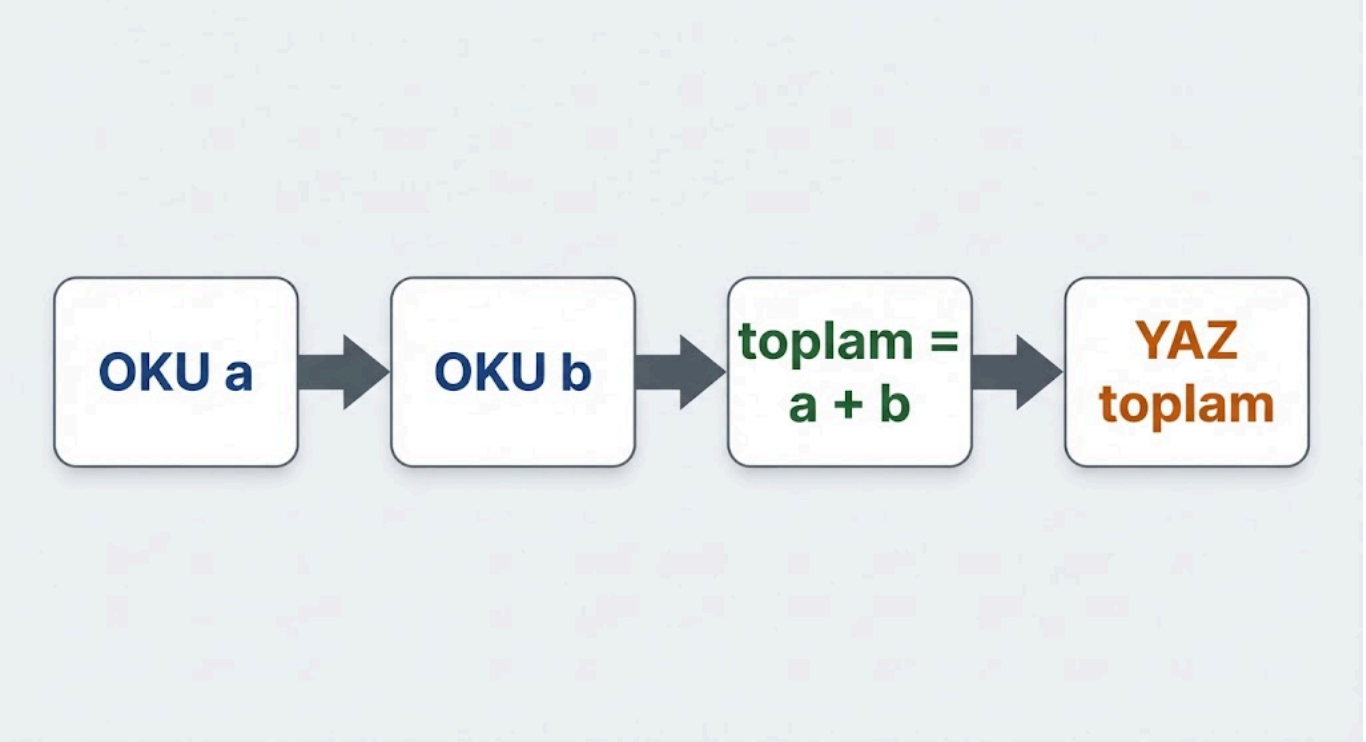
3. SÖZDE KODUN TEMEL YAPILARI

Sözde kodda kullanılan üç temel kontrol yapısı vardır: **sıralı yapı**, **koşullu yapı** ve **döngülü yapı**.

3.1 Sıralı Yapı (Sequential)

Adımlar yukarıdan aşağıya sırayla işlenir.

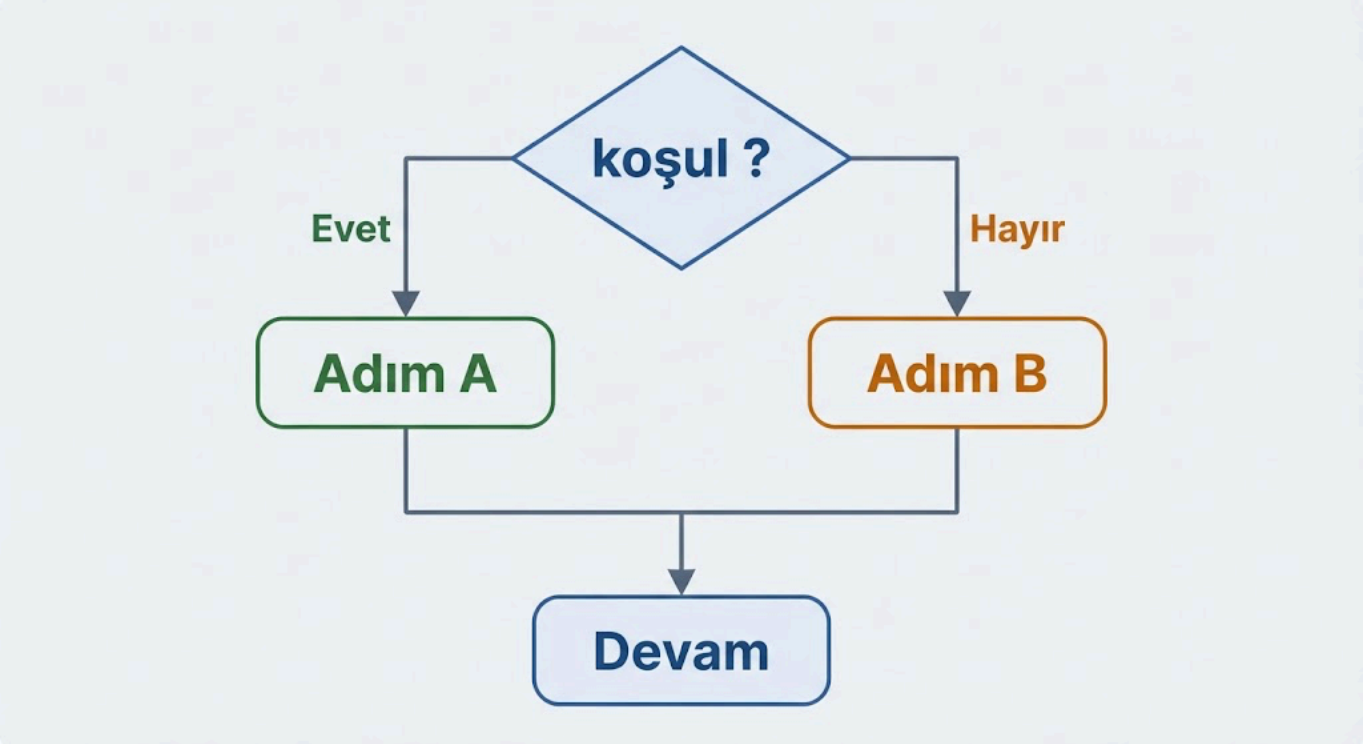
```
BAŞLA
    OKU a
    OKU b
    toplam = a + b
    YAZ toplam
BİTİR
```



3.2 Koşullu Yapı (Conditional)

Belirli bir koşula göre farklı yollar izlenir.

```
EĞER (koşul) İSE
    adımlar
DEĞİLSE
    adımlar
SON EĞER
```

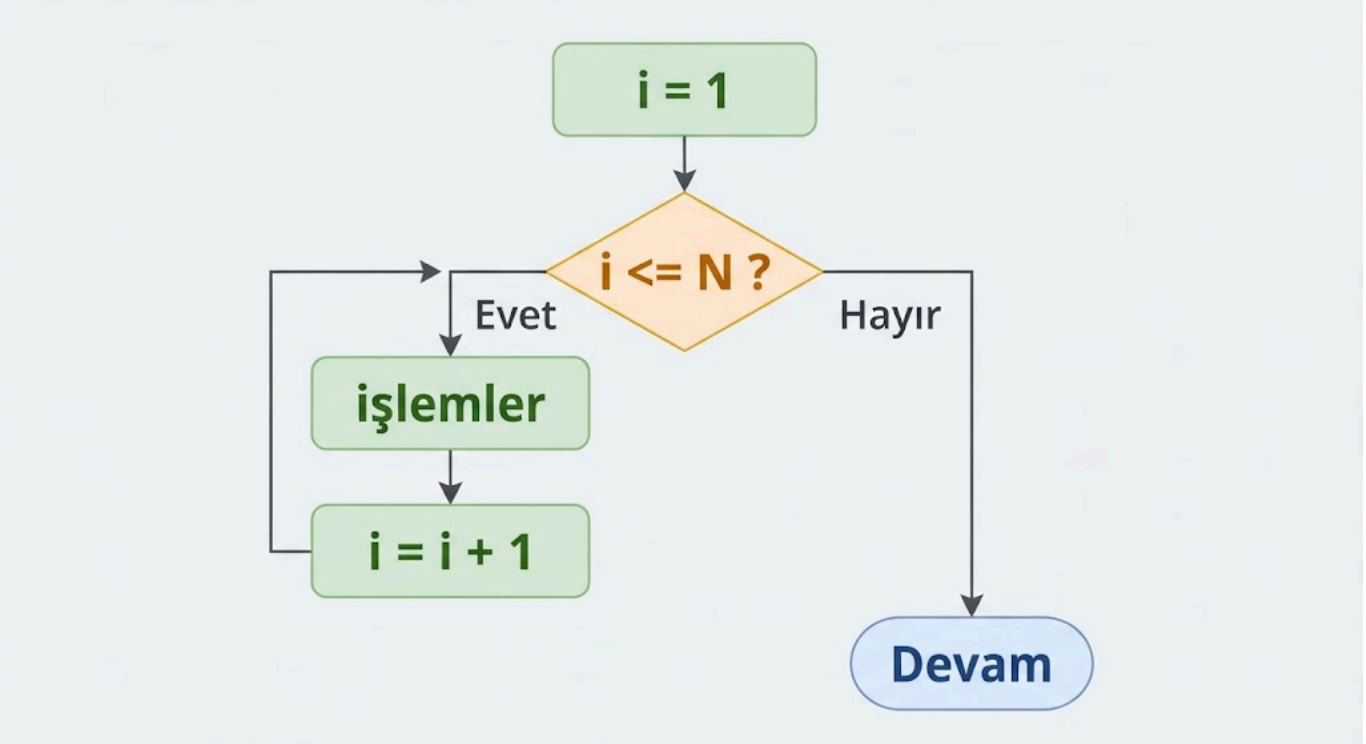


3.3 Döngülü Yapı (Iterative)

Belirli bir koşul sağlandığı sürece adımlar tekrarlanır.

DÖNGÜ ... DÖNGÜ SONU:

```
i = 1
DÖNGÜ (i <= N)
    işlemler
    i = i + 1
DÖNGÜ SONU
```



DIŞINDA ... DIŞINDA DÖNGÜ SONU (while):

```
DÖNGÜ (koşul)
    işlemler
DÖNGÜ SONU
```

3.4 Sözde Kodda Sık Kullanılan Komutlar

Komut	Anlamı
BAŞLA / BİTİR	Algoritmanın başı ve sonu
OKU	Girdi al
YAZ	Çıktı ver
EĞER ... İSE ... DEĞİLSE	Koşul
DÖNGÜ ... DÖNGÜ SONU	Tekrar
DÖN	Döngüden çık
DEVAM ET	Sonraki adıma geç
FONKSİYON / GERİ DÖN	Alt program

---

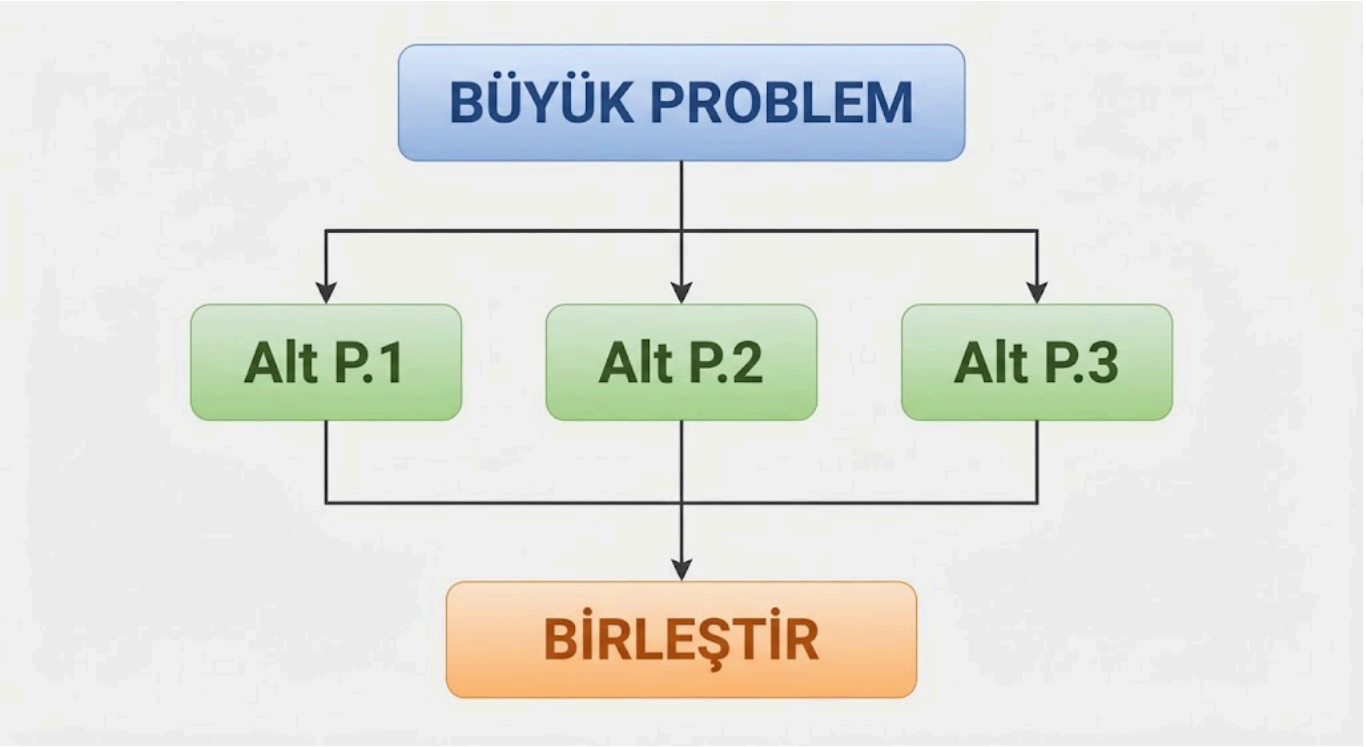
4. ALGORİTMİK DÜŞÜNME TEKNİKLERİ

Algoritmik düşünme, bir problemi sistematik biçimde çözme becerisidir. Aşağıda bu düşünme biçiminin temel teknikleri ve problem çözme stratejileri verilmiştir.

4.1 Temel Algoritmik Düşünme Teknikleri

4.1.1 Parçala ve Yönet (Divide and Conquer)

Problemi daha küçük alt problemlere böl, her birini ayrı ayrı çöz, sonuçları birleştir.



Örnek: Bir diziyi sıralamak için diziyi ikiye böl, her yarıyı sırala, sonra birleştir (Merge Sort).

4.1.2 Adım Adım İlerleme (Stepwise Refinement)

Problemi en genel hâlinden başlayarak adım adım detaylandır.

```
Seviye 1:  Problemi çöz
Seviye 2:  Problemi alt adımlara böl
Seviye 3:  Her alt adımı detaylandır
Seviye 4:  Her detayı sözde koda dök
```

Örnek: "Ortalama hesapla" görevi:

1. Sayıları oku
2. Toplamı bul
3. Sayı adedini bul
4. Toplamı adede böl

4.1.3 Geri İzleme (Backtracking)

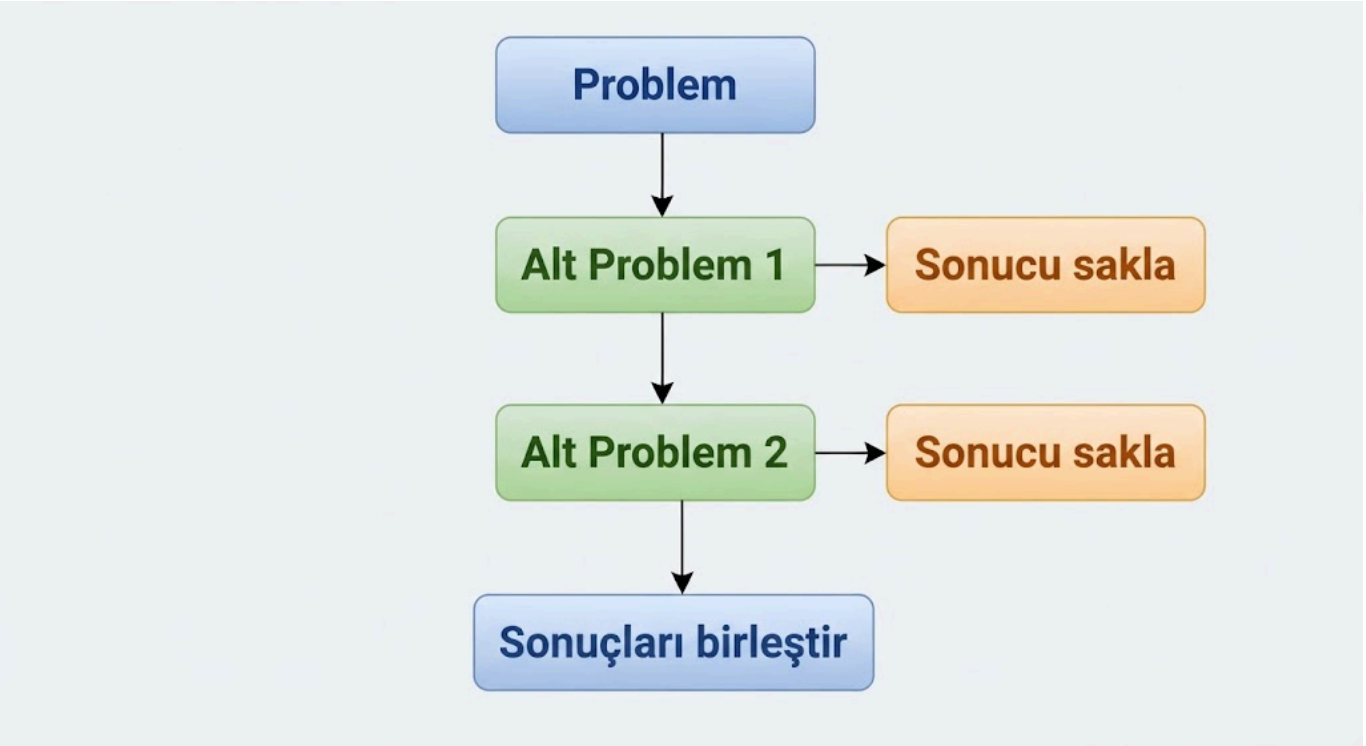
Çözüm yolunda ilerlerken çıkmaza girildiğinde bir önceki adıma dönüp farklı yol dene.



Örnek: Labirent çözme, N-vezir problemi.

4.1.4 Dinamik Programlama (Dynamic Programming)

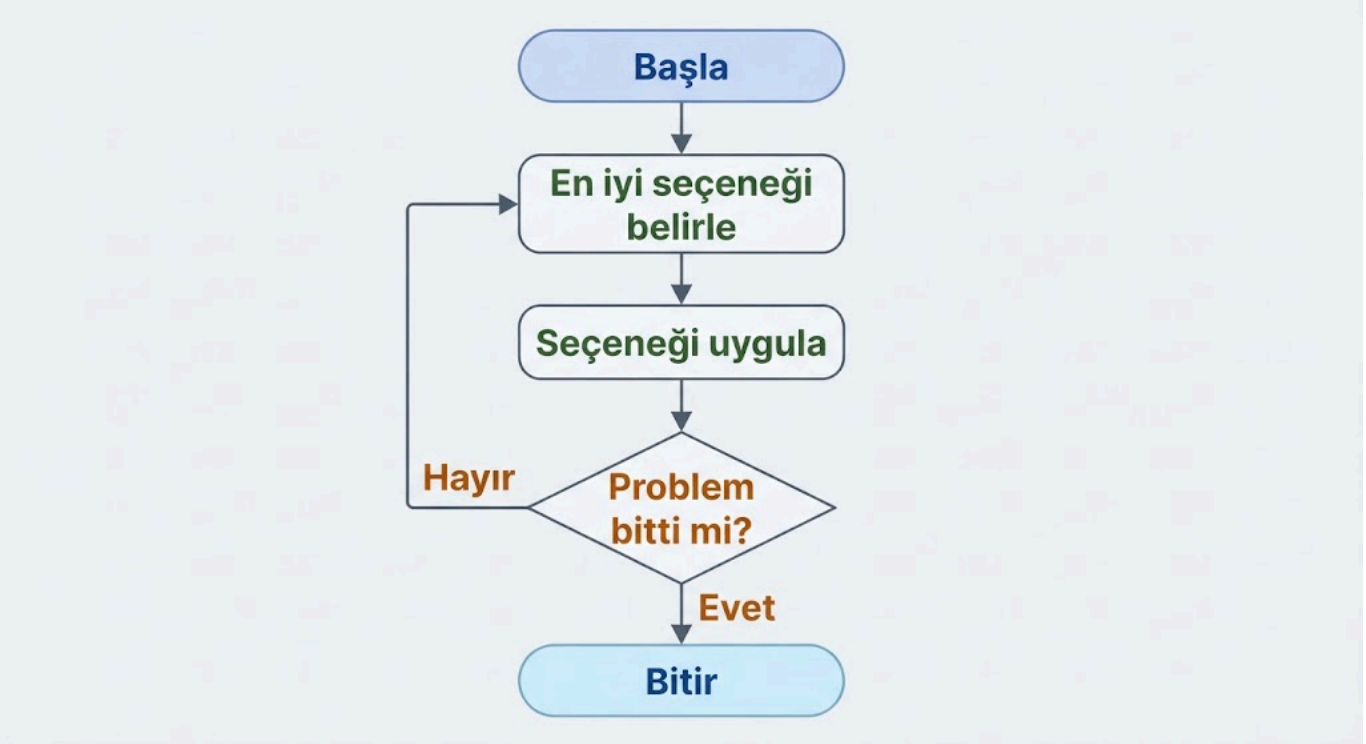
Alt problemlerin sonuçlarını saklayarak aynı işi tekrar yapmaktan kaçın.



Örnek: Fibonacci sayıları, sırt çantası problemi.

4.1.5 Açgözlü Yaklaşım (Greedy)

Her adımda o an için en iyi görünen seçimi yap.



Örnek: Bozuk para üstü verme, en kısa yol (bazı durumlarda).

4.2 Problem Çözme Adımları



4.3 Algoritmik Düşünmeyi Geliştiren Sorular

1. Problem tam olarak ne istiyor?
2. Girdiler neler? Çıktı ne olmalı?
3. Problemi daha küçük parçalara bölebilir miyim?
4. Benzer bir problem daha önce çözüldü mü?
5. Özel durumlar (sıfır, negatif, boş) var mı?

6. En verimli çözüm hangisi?

---

5. UYGULAMA: MATEMATİKSEL PROBLEMLER

Bu bölümde matematiksel problemlerin sözde kod ile nasıl çözüleceği adım adım gösterilecektir.

PROBLEM 1: Faktöriyel Hesaplama

Problem: Girilen N pozitif tam sayısının faktöriyelini (N!) hesaplayan algoritmayı yazınız.

Matematiksel Tanım: N! = 1 × 2 × 3 × ... × N, 0! = 1

Girdi: N
Çıktı: faktoriyel

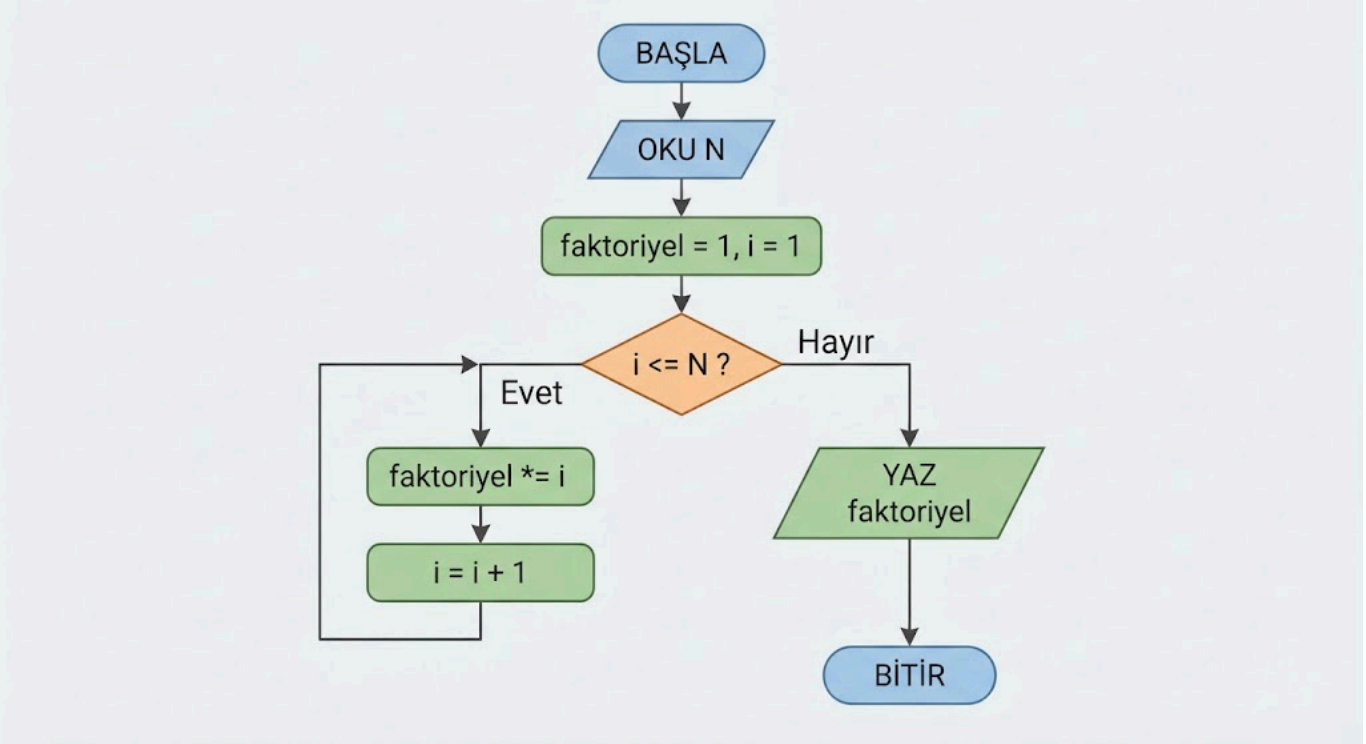
Adım Adım Çözüm

1. Başla.
2. N değerini oku.
3. faktoriyel = 1, i = 1 ata.
4. i ≤ N olduğu sürece:
   - faktoriyel = faktoriyel × i
   - i = i + 1
5. faktoriyel değerini yaz.
6. Bitir.

Sözde Kod

```
BAŞLA
    OKU N
    faktoriyel = 1
    i = 1
    DÖNGÜ (i <= N)
        faktoriyel = faktoriyel * i
        i = i + 1
    DÖNGÜ SONU
    YAZ faktoriyel
BİTİR
```

Akış Diyagramı



Örnek Çalışma (N = 5)

```
i = 1  -->  faktoriyel = 1 * 1 = 1
i = 2  -->  faktoriyel = 1 * 2 = 2
i = 3  -->  faktoriyel = 2 * 3 = 6
i = 4  -->  faktoriyel = 6 * 4 = 24
i = 5  -->  faktoriyel = 24 * 5 = 120
i = 6  -->  döngüden çık
Çıktı : 120
```

PROBLEM 2: Asal Sayı Kontrolü

Problem: Girilen N sayısının asal olup olmadığını bulan algoritmayı yazınız.

Tanım: Asal sayı, yalnızca 1'e ve kendisine bölünebilen, 1'den büyük doğal sayıdır.

Girdi: N
Çıktı: "Asal" veya "Asal Değil"

Adım Adım Çözüm

1. Başla.
2. N değerini oku.
3. Eğer N < 2 ise "Asal Değil" yaz, bitir.
4. i = 2 ata.
5. i ≤ N/2 olduğu sürece:
    - Eğer N mod i = 0 ise "Asal Değil" yaz, bitir.
    - i = i + 1

6. "Asal" yaz.
7. Bitir.

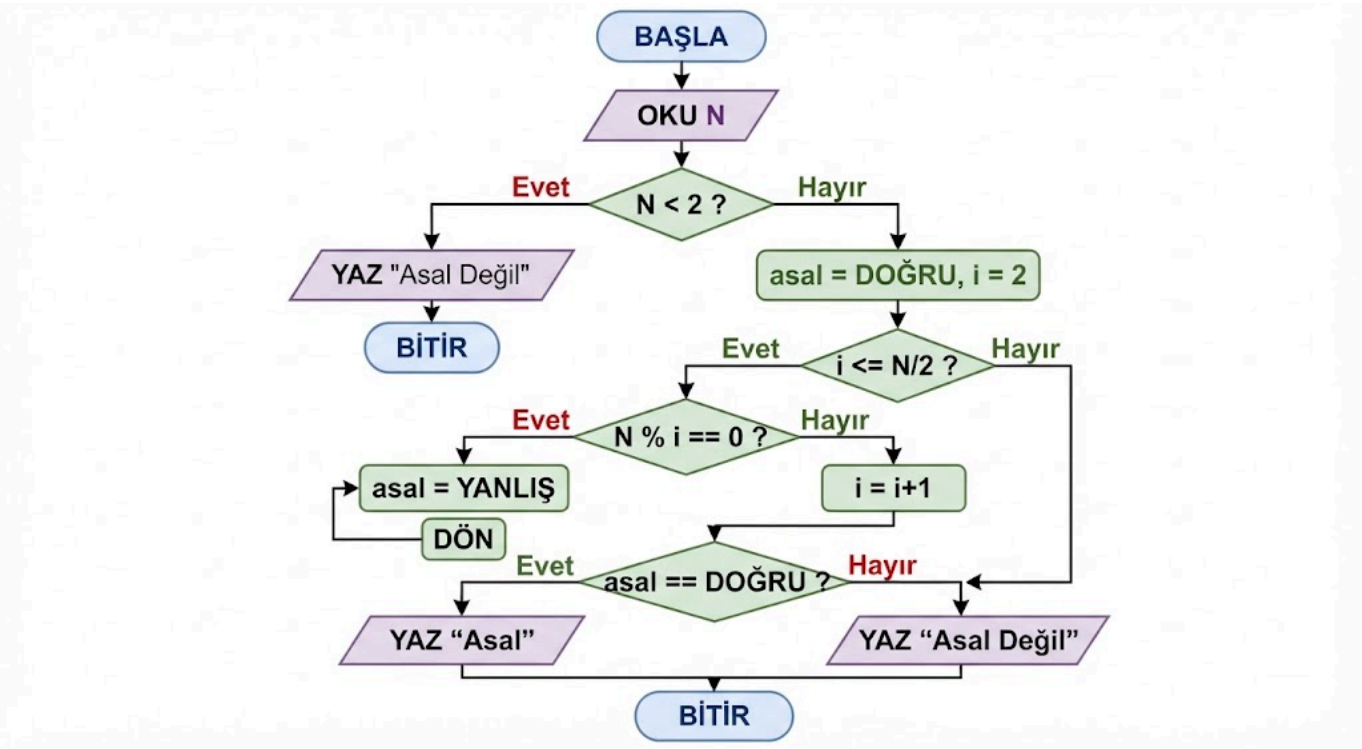
Sözde Kod

```
BAŞLA
    OKU N
    EĞER (N < 2) İSE
        YAZ "Asal Değil"
        BİTİR
    SON EĞER

    asal = DOĞRU
    i = 2
    DÖNGÜ (i <= N / 2)
        EĞER (N % i == 0) İSE
            asal = YANLIŞ
            DÖN
        SON EĞER
        i = i + 1
    DÖNGÜ SONU

    EĞER (asal == DOĞRU) İSE
        YAZ "Asal"
    DEĞİLSE
        YAZ "Asal Değil"
    SON EĞER
BİTİR
```

Akış Diyagramı



Örnek Çalışma

```
N = 7:
  i = 2  -->  7 % 2 = 1  -->  devam
  i = 3  -->  7 % 3 = 1  -->  devam
  i = 4  -->  7 % 4 = 3  -->  devam (i > N/2 = 3.5 oldu, çık)
  Çıktı: Asal

N = 9:
  i = 2  -->  9 % 2 = 1  -->  devam
  i = 3  -->  9 % 3 = 0  -->  asal = YANLIŞ, DÖN
  Çıktı: Asal Değil
```

PROBLEM 3: En Büyük Ortak Bölen (EBOB) — Öklid Algoritması

Problem: İki pozitif tam sayının EBOB'unu bulan algoritmayı yazınız.

Tanım: EBOB(a, b) = a ve b'yi tam bölen en büyük sayı.

Öklid Yöntemi: EBOB(a, b) = EBOB(b, a mod b), EBOB(a, 0) = a

Girdi: a, b
Çıktı: EBOB

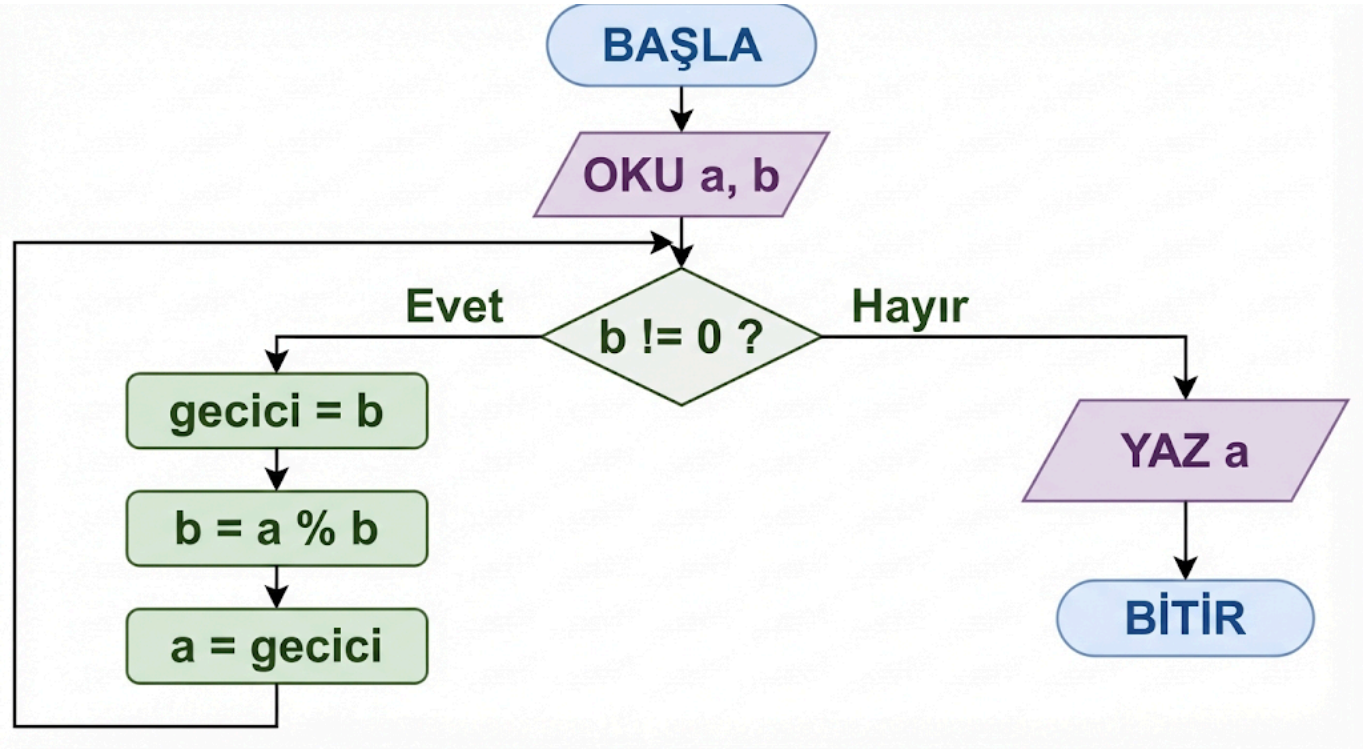
Adım Adım Çözüm

1. Başla.
2. a ve b değerlerini oku.
3. b ≠ 0 olduğu sürece:
    - gecici = b
    - b = a mod b
    - a = gecici
4. a değerini yaz.
5. Bitir.

Sözde Kod

```
BAŞLA
    OKU a
    OKU b
    DÖNGÜ (b != 0)
        gecici = b
        b = a % b
        a = gecici
    DÖNGÜ SONU
    YAZ a
BİTİR
```

Akış Diyagramı



Örnek Çalışma (a = 48, b = 18)

```
Adım 1: gecici = 18, b = 48 % 18 = 12, a = 18
Adım 2: gecici = 12, b = 18 % 12 = 6,  a = 12
Adım 3: gecici = 6,  b = 12 % 6 = 0,   a = 6
Adım 4: b = 0, döngüden çık
Çıktı : 6
```

PROBLEM 4: Bir Sayının Basamakları Toplamı

Problem: Girilen pozitif tam sayının basamaklarındaki rakamların toplamını bulan algoritmayı yazınız.

Girdi: N
Çıktı: basamak_toplami

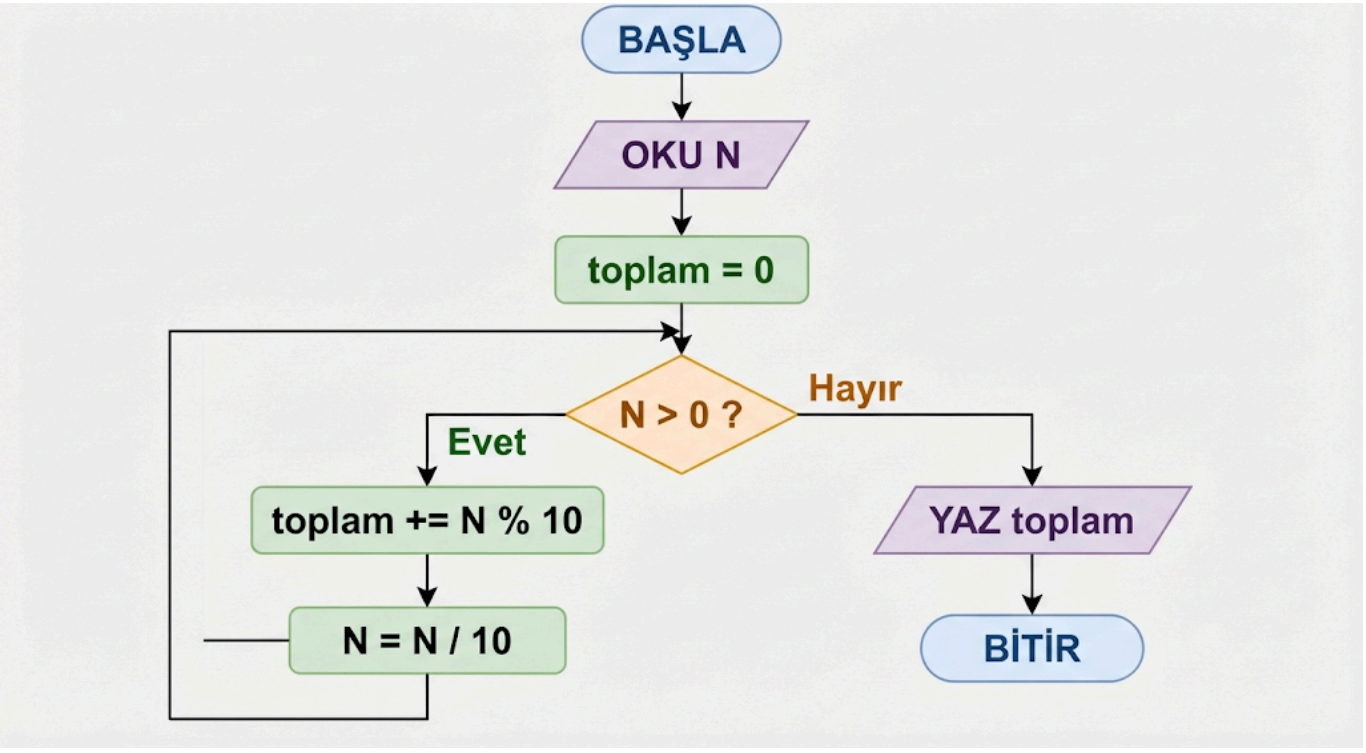
Adım Adım Çözüm

1. Başla.
2. N değerini oku.
3. toplam = 0 ata.
4. N > 0 olduğu sürece:
    - toplam = toplam + (N mod 10)
    - N = N / 10 (tam sayı bölmesi)
5. toplam değerini yaz.
6. Bitir.

Sözde Kod

```
BAŞLA
    OKU N
    toplam = 0
    DÖNGÜ (N > 0)
        toplam = toplam + (N % 10)
        N = N / 10
    DÖNGÜ SONU
    YAZ toplam
BİTİR
```

Akış Diyagramı



Örnek Çalışma (N = 4725)

```
N = 4725:  toplam = 0 + 5 = 5,   N = 472
N = 472 :  toplam = 5 + 2 = 7,   N = 47
N = 47  :  toplam = 7 + 7 = 14,  N = 4
N = 4   :  toplam = 14 + 4 = 18, N = 0
N = 0   :  döngüden çık
Çıktı : 18
```

PROBLEM 5: Fibonacci Dizisi

Problem: İlk N Fibonacci sayısını ekrana yazdıran algoritmayı yazınız.

Tanım: F(0) = 0, F(1) = 1, F(n) = F(n-1) + F(n-2)

Girdi: N
Çıktı: İlk N Fibonacci sayısı

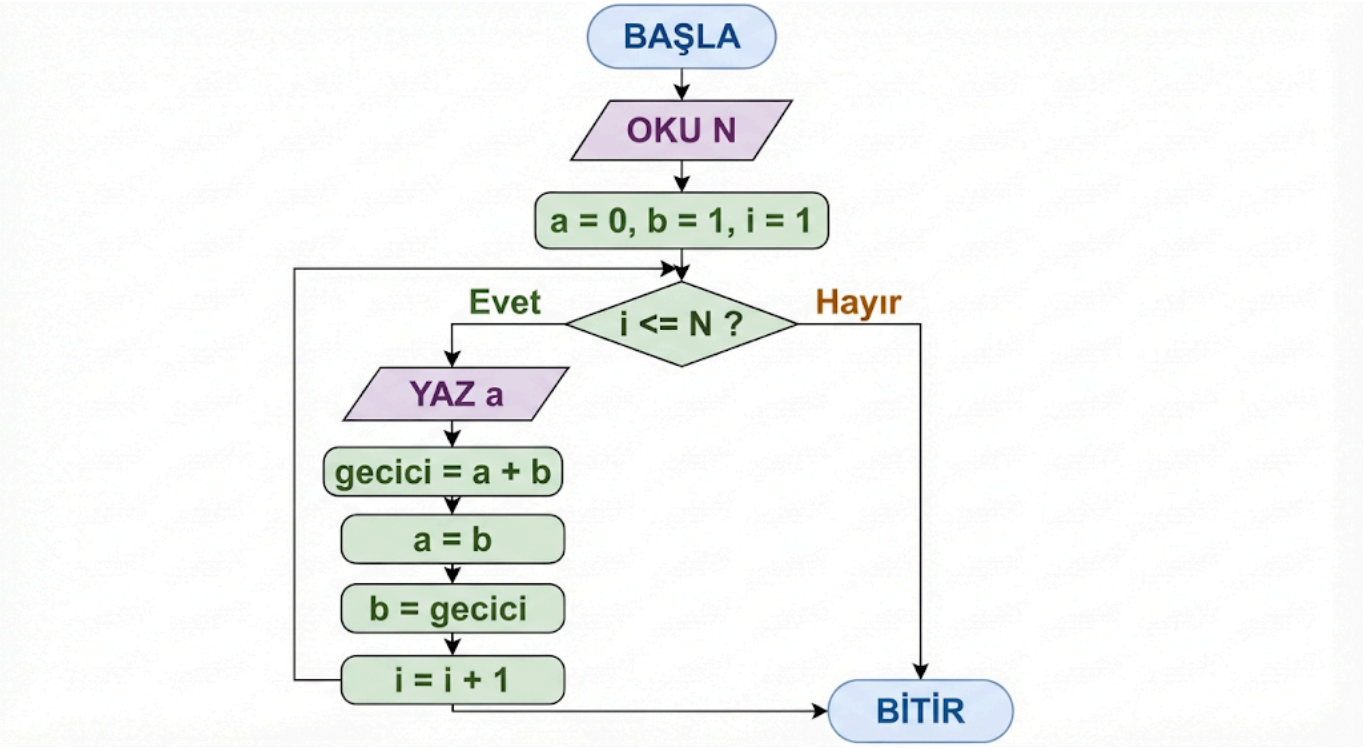
Adım Adım Çözüm

1. Başla.
2. N değerini oku.
3. a = 0, b = 1 ata.
4. i = 1'den N'e kadar:
    - a değerini yaz.
    - gecici = a + b
    - a = b
    - b = gecici
5. Bitir.

Sözde Kod

```
BAŞLA
    OKU N
    a = 0
    b = 1
    i = 1
    DÖNGÜ (i <= N)
        YAZ a
        gecici = a + b
        a = b
        b = gecici
        i = i + 1
    DÖNGÜ SONU
BİTİR
```

Akış Diyagramı



Örnek Çalışma (N = 8)

```
i = 1: YAZ 0;  gecici = 1;  a = 1;  b = 1
i = 2: YAZ 1;  gecici = 2;  a = 1;  b = 2
i = 3: YAZ 1;  gecici = 3;  a = 2;  b = 3
i = 4: YAZ 2;  gecici = 5;  a = 3;  b = 5
i = 5: YAZ 3;  gecici = 8;  a = 5;  b = 8
i = 6: YAZ 5;  gecici = 13; a = 8;  b = 13
i = 7: YAZ 8;  gecici = 21; a = 13; b = 21
i = 8: YAZ 13; döngüden çık
Çıktı: 0 1 1 2 3 5 8 13
```

PROBLEM 6: İki Sayının En Küçük Ortak Katı (EKOK)

Problem: İki pozitif tam sayının EKOK'unu bulan algoritmayı yazınız.

Formül: EKOK(a, b) = (a × b) / EBOB(a, b)

Girdi: a, b
Çıktı: EKOK

Adım Adım Çözüm

1. Başla.
2. a ve b değerlerini oku.
3. a ve b'nin çarpımını sakla: carpim = a * b
4. EBOB'u bul (Öklid algoritması).
5. ekok = carpim / ebob

6. ekok değerini yaz.
7. Bitir.

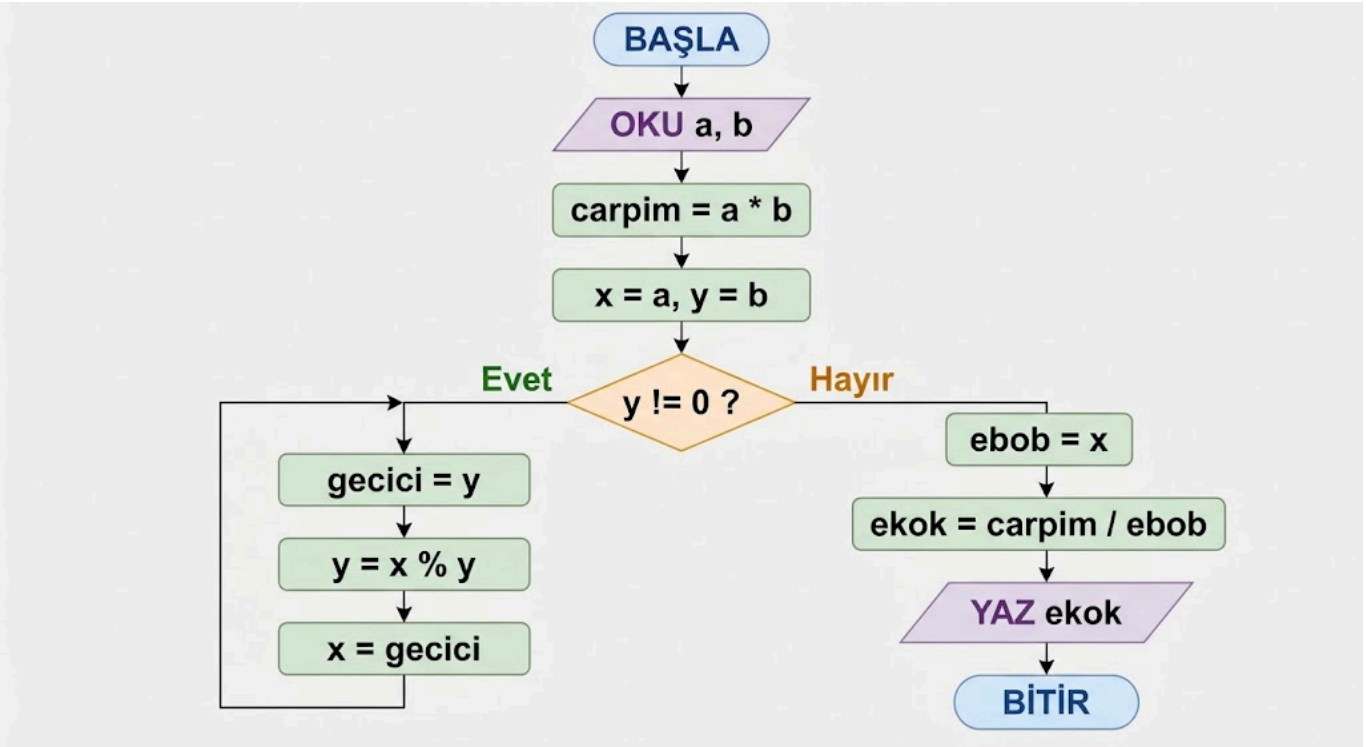
Sözde Kod

```
BAŞLA
    OKU a
    OKU b
    carpim = a * b

    x = a
    y = b
    DÖNGÜ (y != 0)
        gecici = y
        y = x % y
        x = gecici
    DÖNGÜ SONU

    ebob = x
    ekok = carpim / ebob
    YAZ ekok
BİTİR
```

Akış Diyagramı



Örnek Çalışma (a = 12, b = 18)

```
carpim = 12 * 18 = 216
EBOB(12, 18):
  x = 12, y = 18
  gecici = 18, y = 12 % 18 = 12, x = 18
  gecici = 12, y = 18 % 12 = 6,  x = 12
  gecici = 6,  y = 12 % 6 = 0,   x = 6
  y = 0, çık
ebob = 6
ekok = 216 / 6 = 36
Çıktı: 36
```

PROBLEM 7: Sayı Tahmin Oyunu

Problem: Bilgisayarın 1 ile 100 arasında rastgele tuttuğu sayıyı kullanıcının tahmin etmeye çalıştığı algoritmayı yazınız. Kullanıcıya "daha büyük" veya "daha küçük" ipuçları verilir.

Girdi: tahmin (kullanıcıdan)
Çıktı: "Doğru", "Daha Büyük", "Daha Küçük"

Adım Adım Çözüm

1. Başla.
2. Bilgisayar 1-100 arası rastgele bir sayı tut: hedef
3. sayac = 0 ata.
4. Döngü başlat:
    - tahmin değerini oku.
    - sayac = sayac + 1
    - Eğer tahmin = hedef ise "Doğru" yaz ve döngüden çık.
    - Eğer tahmin < hedef ise "Daha Büyük" yaz.
    - Eğer tahmin > hedef ise "Daha Küçük" yaz.
5. sayac değerini yaz.
6. Bitir.

Sözde Kod

```
BAŞLA
    hedef = RASTGELE(1, 100)
    sayac = 0
    bulundu = YANLIŞ

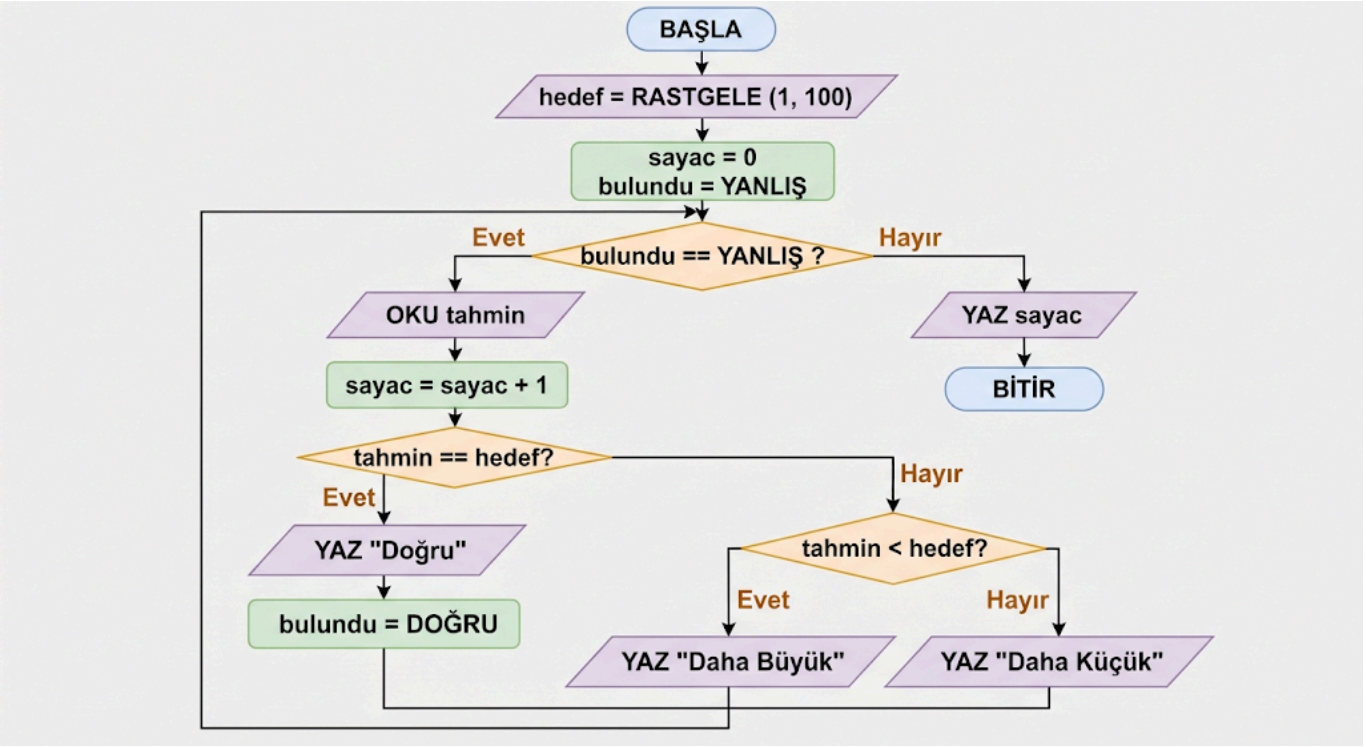
    DÖNGÜ (bulundu == YANLIŞ)
        OKU tahmin
        sayac = sayac + 1

        EĞER (tahmin == hedef) İSE
            YAZ "Doğru"
            bulundu = DOĞRU
```

```
        DEĞİLSE EĞER (tahmin < hedef) İSE
            YAZ "Daha Büyük"
        DEĞİLSE
            YAZ "Daha Küçük"
        SON EĞER
    DÖNGÜ SONU

    YAZ sayac
BİTİR
```

Akış Diyagramı



Örnek Çalışma (hedef = 42)

```
Tahmin: 50  -->  Daha Küçük  (sayac = 1)
Tahmin: 25  -->  Daha Büyük  (sayac = 2)
Tahmin: 37  -->  Daha Büyük  (sayac = 3)
Tahmin: 44  -->  Daha Küçük  (sayac = 4)
Tahmin: 41  -->  Daha Büyük  (sayac = 5)
Tahmin: 42  -->  Doğru       (sayac = 6)
Çıktı: 6 denemede bulundu
```

---

6. SÖZDE KOD YAZIM KURALLARI

Sözde kod yazarken dikkat edilmesi gereken temel kurallar:

```
+----------------------------------+--------------------------------------
--+
| Kural                            | Açıklama
|
+----------------------------------+--------------------------------------
--+
| Anlaşılır isimler kullan         | a, b yerine sayi1, toplam gibi
|
| Girintileme yap                  | Döngü ve koşul içleri girintili
olmalı |
| Yorum ekle                       | Karmaşık adımları açıkla
|
| Tek bir işlem yap                | Her satırda bir işlem
|
| Büyük/küçük harf tutarlı ol      | Tüm anahtar kelimeler aynı biçimde
|
| Sonlanma noktası belirt          | BİTİR ile bitir
|
| Gereksiz ayrıntıdan kaçın        | Dil kurallarına girme
|
+----------------------------------+--------------------------------------
--+
```

İyi ve Kötü Sözde Kod Karşılaştırması

Kötü:

```
x = oku
y = oku
z = x + y
yaz z
```

İyi:

```
BAŞLA
    OKU sayi1
    OKU sayi2
    toplam = sayi1 + sayi2
    YAZ toplam
BİTİR
```

---

7. SÖZDE KODDAN GERÇEK KODA GEÇİŞ

Sözde kod, programlama diline çevrilirken yapı korunur, yalnızca sözdizimi değişir.

Sözde Kod:

```
BAŞLA
    OKU N
    faktoriyel = 1
    i = 1
    DÖNGÜ (i <= N)
        faktoriyel = faktoriyel * i
        i = i + 1
    DÖNGÜ SONU
    YAZ faktoriyel
BİTİR
```

Python Karşılığı:

```python
N = int(input("N: "))
faktoriyel = 1
i = 1
while i <= N:
    faktoriyel *= i
    i += 1
print(faktoriyel)
```

C Karşılığı:

```c
int N, faktoriyel = 1, i = 1;
scanf("%d", &N);
while (i <= N) {
    faktoriyel *= i;
    i++;
}
printf("%d", faktoriyel);
```

Görüldüğü gibi mantık aynıdır; yalnızca yazım biçimi değişir.

---

8. ÖZET

- **Sözde kod**, algoritmanın dilden bağımsız, okunabilir biçimde yazılmasıdır.
- Üç temel yapı içerir: **sıralı, koşullu, döngülü**.
- **Algoritmik düşünme teknikleri**: parçala-yönet, adım adım ilerleme, geri izleme, dinamik programlama, açgözlü yaklaşım.
- Problem çözme adımları: **anla → girdi/çıktı belirle → planla → sözde kod yaz → test et → düzelt → iyileştir**.
- Matematiksel problemler (faktöriyel, asal sayı, EBOB, EKOK, basamak toplamı, Fibonacci, tahmin oyunu) sözde kod ve akış diyagramları ile çözülmüştür.
- Sözde kod, gerçek koda geçişte **köprü** görevi görür.

---

9. DEĞERLENDİRME SORULARI

1. Sözde kod nedir? Gerçek koddan farklarını maddeler hâlinde yazınız.
2. Sözde kodda kullanılan üç temel kontrol yapısını örneklerle açıklayınız.
3. Algoritmik düşünme tekniklerinden "parçala ve yönet" ile "açgözlü yaklaşım" arasındaki farkı açıklayınız.
4. Girilen bir sayının basamak sayısını bulan algoritmayı sözde kod ve akış diyagramı ile yazınız.
5. 1'den 100'e kadar olan çift sayıların toplamını bulan algoritmayı sözde kod ile yazınız.
6. Girilen üç sayının en büyüğünü bulan algoritmayı sözde kod ve akış diyagramı ile gösteriniz.
7. Bir sayının tersini (örneğin 1234 → 4321) bulan algoritmayı sözde kod ile yazınız.
8. Sözde kod yazarken dikkat edilmesi gereken kuralları maddeler hâlinde belirtiniz.